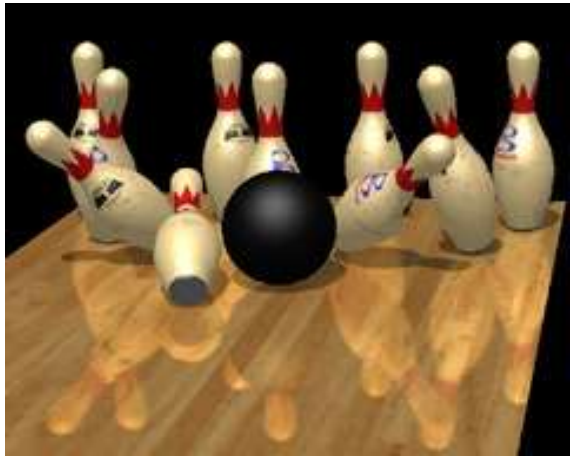
OBJECT MODELLING

Texture mapping: Motivation

Modeling the fine detail in real objects with triangles is just too hard sometimes.

Added surface detail can help computer graphics images look more real.

Texture mapping let's us use real images in our CG scenes to add realistic fine detail without the high geometric costs.



Texture mapping: Fundamentals

Textures are most often 2D images

- A single element of texture is called a texel

- The value of the texel is used to modify surface appearance in some way

- The mapping between texture and surface determines how pixels and texels correspond

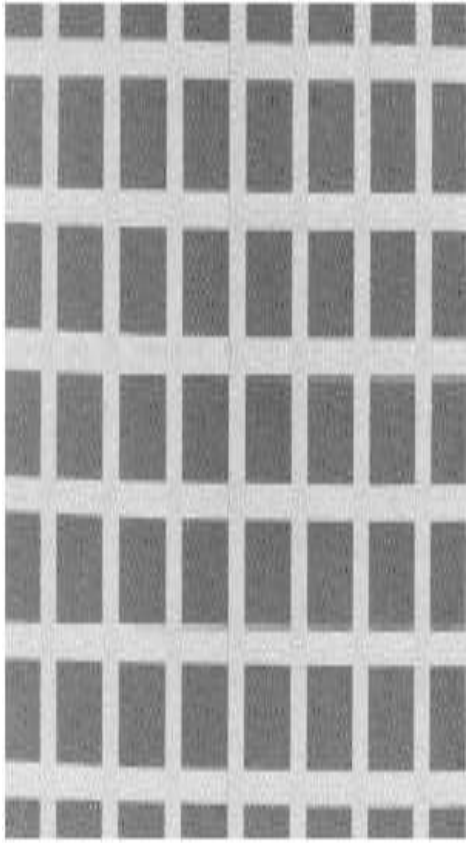
- Texture : A detailed pattern that is repeated many times to tile the plane.

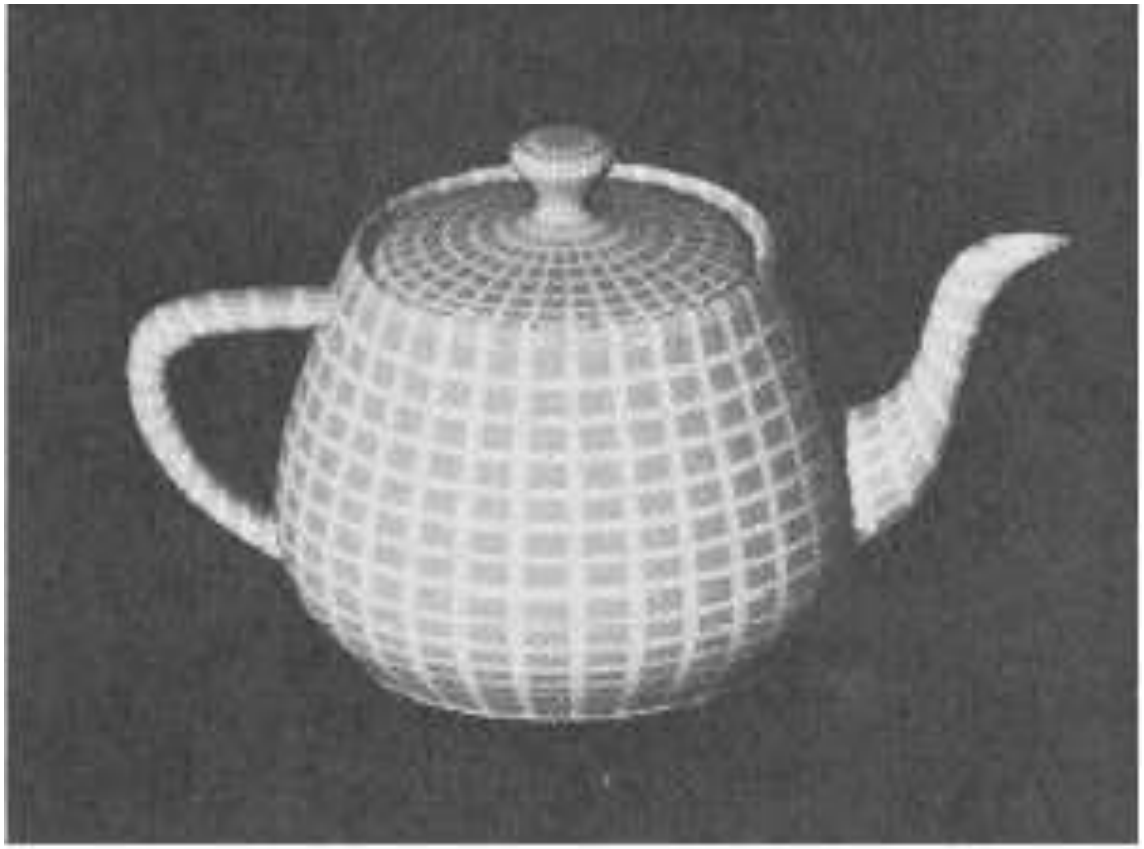
Texture Mapping Basic Strategy

Three steps to applying a texture

1. Specify the texture
   - read or generate image
   - assign to texture
   - enable texturing
2. Assign texture coordinates to vertices
   - Proper mapping function is left to application
3. Specify texture parameters
   - wrapping, filtering

Texture Mapped on to Tea Pot





Types of Texture Mapping

- Two Types
  1. **Forward texture mapping**:

     Computing 3D positions of the texture points and then projecting them onto the image plane.

  2. **Inverse texture mapping**:

     selecting every pixel in the image plane and finding what point of the texture plane is projected there.

Drawbacks

- Forward Texture mapping :

  Adjacent texture points may project onto non adjacent image points, thus creating a non colored area .

- Inverse Texture Mapping :

Aliasing(the stair-stepped appearance of diagonal lines when there are not enough pixels in the image or on screen to represent them realistically) would appear, if sampling the texture map is done at a frequency higher than its Nyquist rate.

Basic Illumination

Light Source Independent Models

Depth Shading

- Color or intensity determined solely by "depth" of polygon.
- Darker colors or intensities at lower elevations.
- Effective in modeling terrain or surface data
- Avoids complex calculations of lighting dependent models
- Simulates realism

Depth Cueing

- Reduce intensity of pixel as the distance from the observer increases
- Simulates reduction in clarity as distances from the observer increases
- Image fades in the distance
- Often used in medical imaging

Light Source Dependent Models

What an object looks like depends on

- Properties of the light source such as color, distance from object, direction from object, intensity of source
- Surface characteristics of object such as color and reflectance properties
- Location of the observer

Light striking a surface of an object can be

- Reflected (Diffuse reflection & Specular reflection)
- Absorbed
- Transmitted (Translucent or transparent)
- Combination of all three

Illumination (Lighting) Model

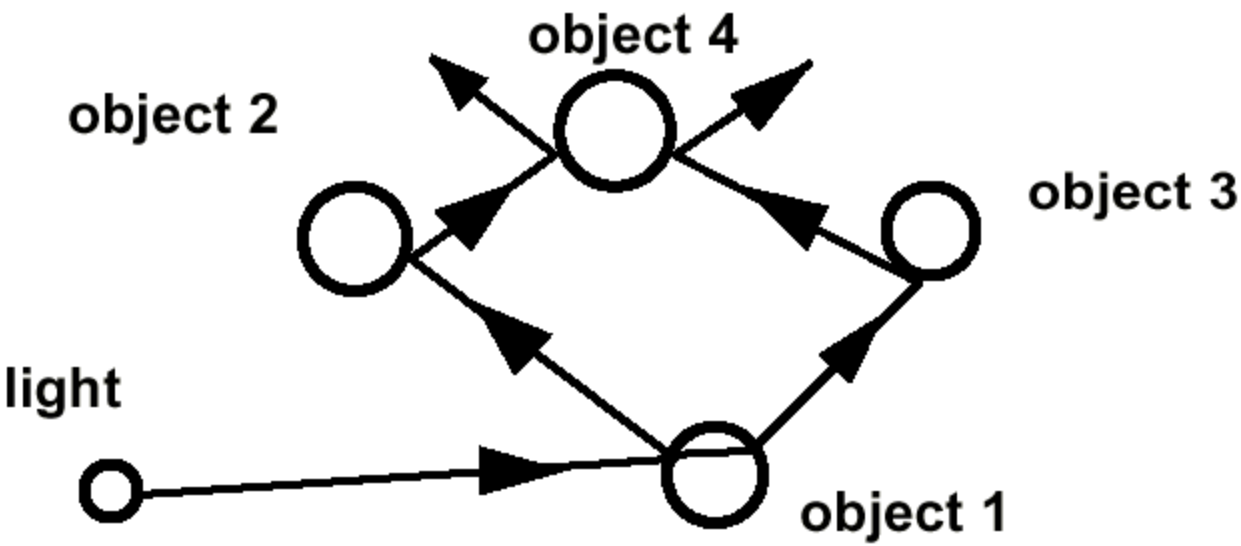
- Illumination (lighting) model: determine the color of a surface point by simulating some light attributes.

- Shading model: applies the illumination models at a set of points and colors the whole image.

- To model the interaction of light with surfaces to determine the final color & brightness of the surface, two models can be used:
  - Global illumination
  - Local illumination

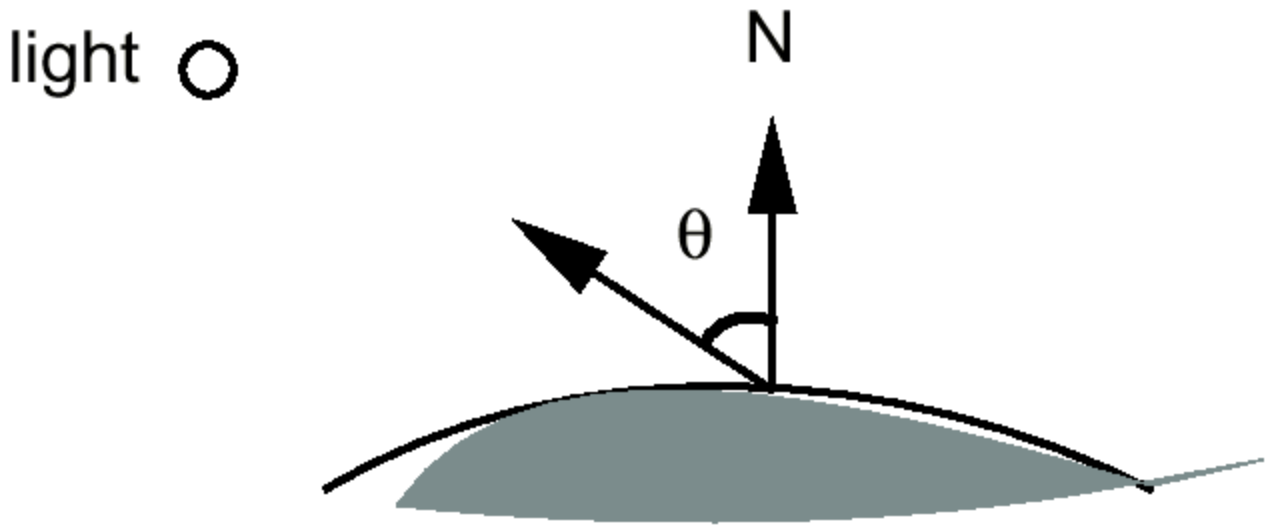
Global Illumination

- Global Illumination models: take into account the interaction of light from all the surfaces in the scene.



Local illumination

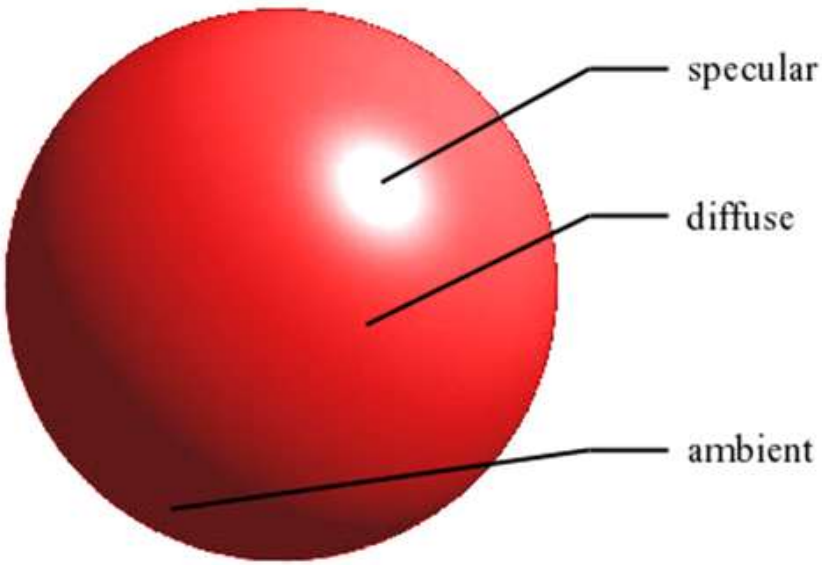
- Only consider the light, the observer position, and the object material properties



Basic Illumination Model

- Simple and fast method for calculating surface intensity at a given point
- Lighting calculation are based on:
  - The background lighting conditions
  - The light source specification: color, position
  - Optical properties of surfaces:
    - Glossy OR matte
    - Opaque OR transparent (control refection and absorption)

Components of Reflections

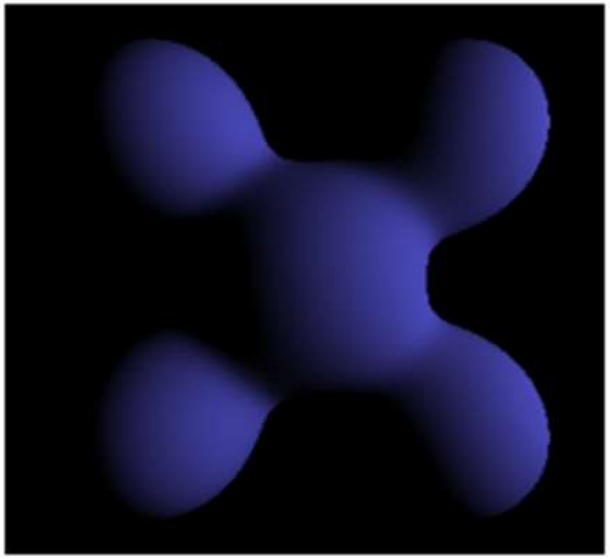


Ambient – surface exposed to **indirect light** reflected from nearby objects.

Diffuse – reflection from **incident** light with equal intensity in all directions. Depends on surface properties.

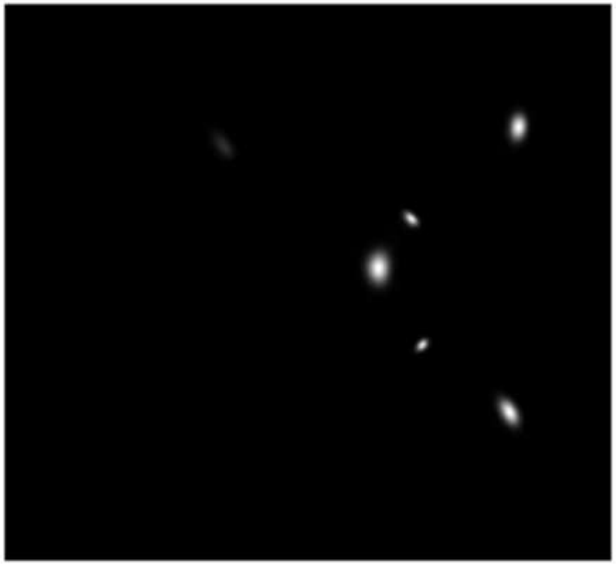
Specular – near **total** of the incident light around reflection angle.

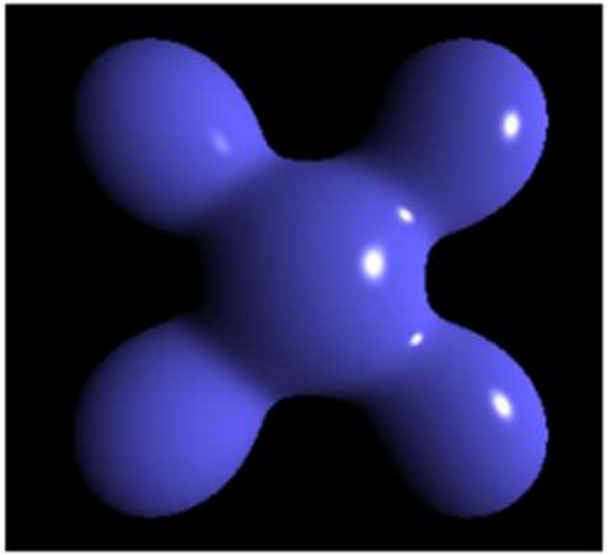
ambient



diffuse

specular





final

Hidden Surface Removal

Visibility

- Assumption: All polygons are **opaque**

- What polygons are visible with respect to your view frustum?
  - Outside: **View Frustum Clipping**
    - Remove polygons outside of the view volume

  - Inside: **Hidden Surface Removal**
    - Backface culling
      - Polygons facing away from the viewer
    - Occlusion
      - Polygons farther away are obscured by closer polygons

- Why should we remove these polygons?
    - Avoid unnecessary expensive operations on these polygons later

Visibility of primitives

- We don’t want to waste time rendering primitives which don’t contribute to the final image.
- A scene primitive can be invisible for 3 reasons:
  - Primitive lies outside field of view
  - Primitive is *back-facing* (under certain conditions)
  - Primitive is occluded by one or more objects nearer the viewer
- How do we remove these efficiently?
- How do we identify these efficiently?

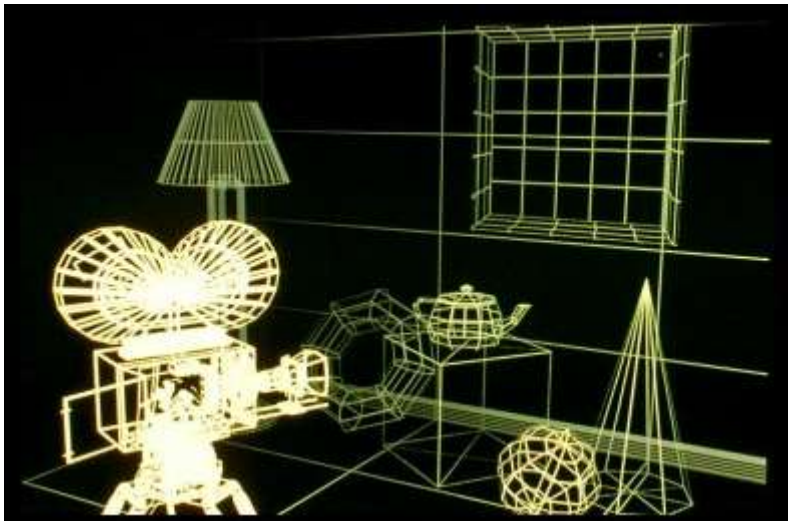
The visibility problem.

Clipping: Remove polygons outside of the view volume

Two problems remain:

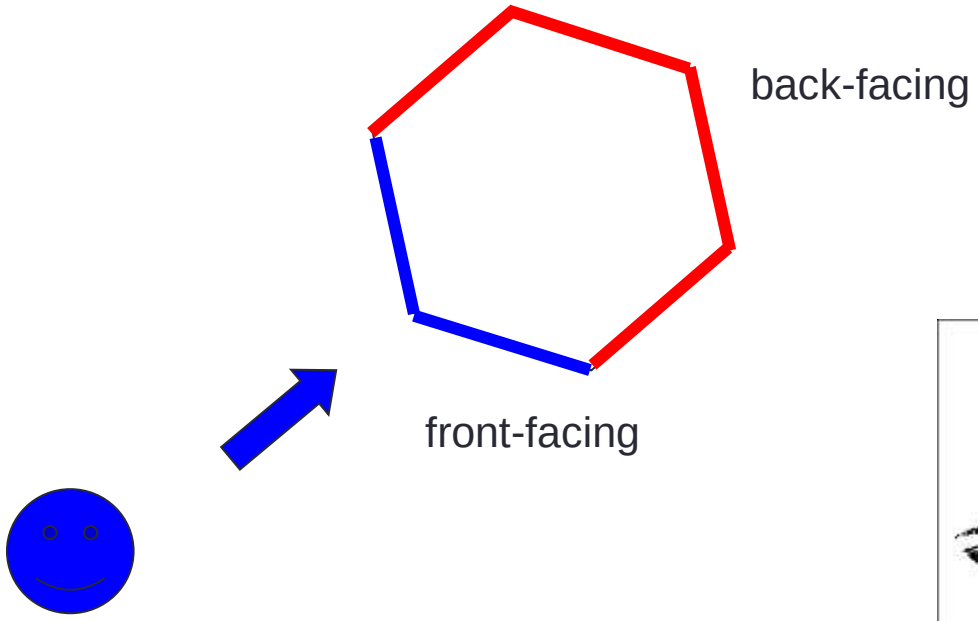
Removal of faces facing away from the viewer.

Removal of faces obscured by closer objects.

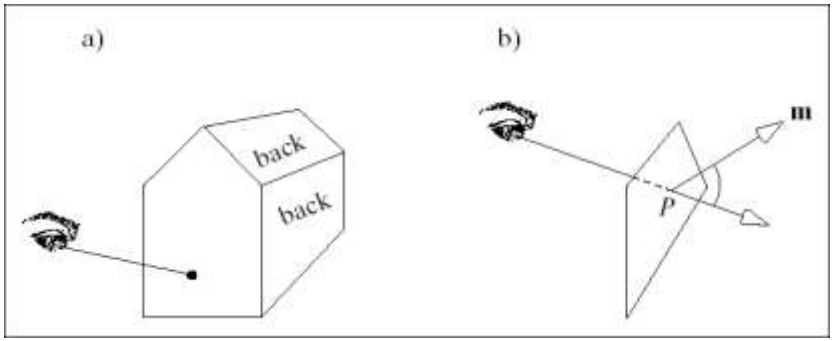


Backface Culling

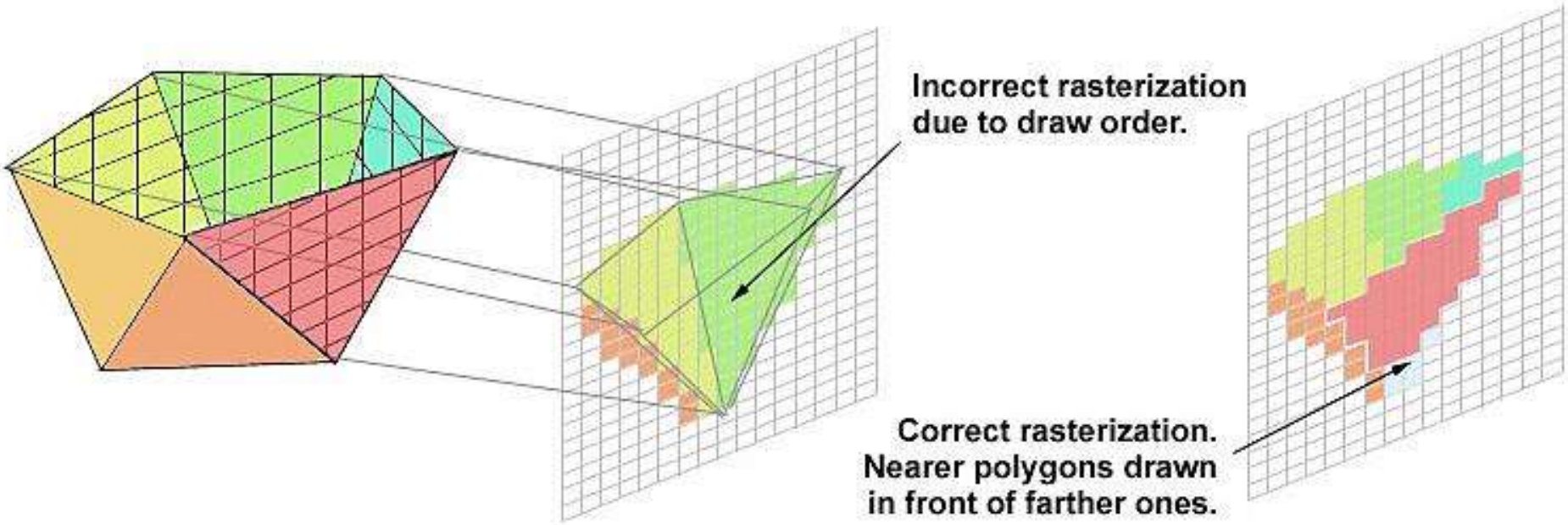
- Avoid drawing polygons facing away from the viewer
  - Front-facing polygons occlude these polygons in a closed polyhedron
- Test if a polygon is front- or back-facing?



Ideas?



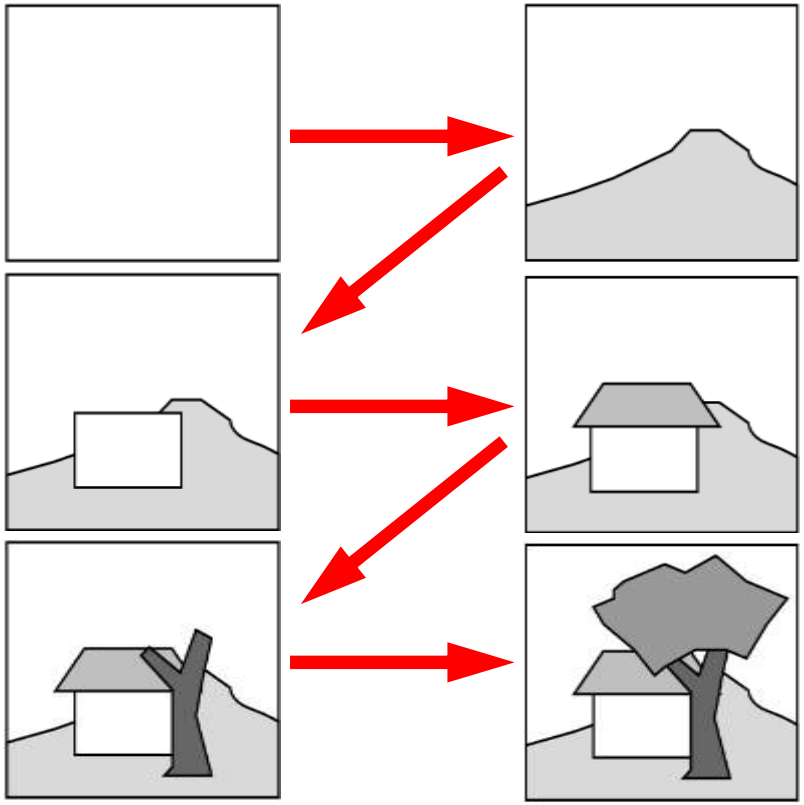
Depth Sorting



Given a collection of polygons:
What order do we draw them in?

Depth Sorting: Painter's Algorithm

- Main Idea
  - A painter creates a picture by drawing background scene elements before foreground ones
- Requirements
  - Draw polygons in back-to-front order
  - Need to **sort** the polygons by depth order to get a correct image

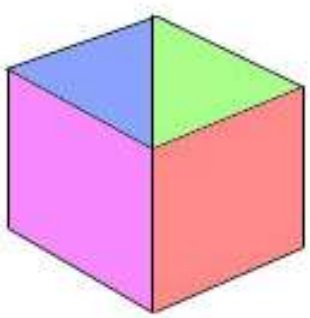


from Shirley

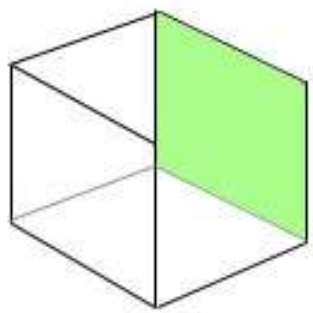
Depth Sorting:
Painter's Algorithm

Method:

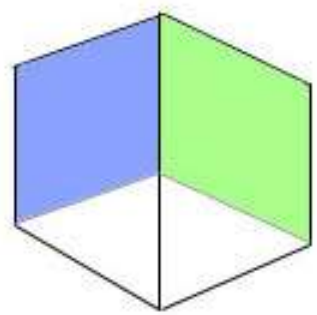
1. Sort polygons from back to front.
2. Draw each sorted polygon, painting over the pixels of existing ones.



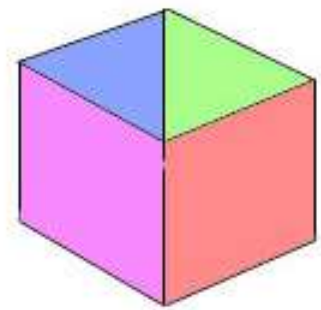
Example: Open Cube.



Paint farthest polygon first.



Next polygon in back-to-front order.

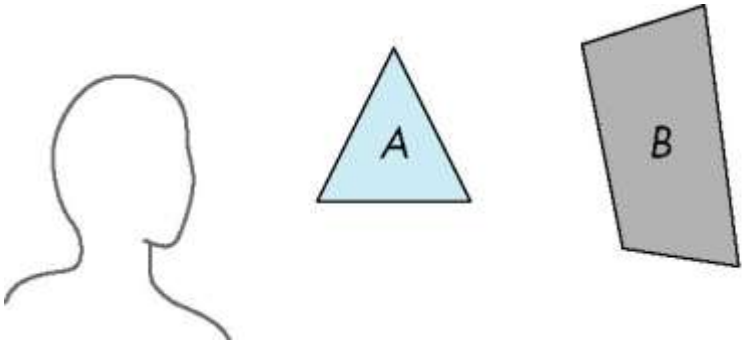


Closest polygons last, overwrite existing ones.

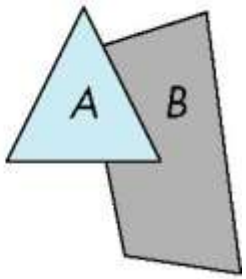
- Requires sorting of polygons for each rendering.
- Which point is used for sorting? Center of polygon is typical.
- Easy to implement

Painter's Algorithm

- Render polygons in back to front order so that polygons behind others are simply painted over



B behind A as seen by viewer

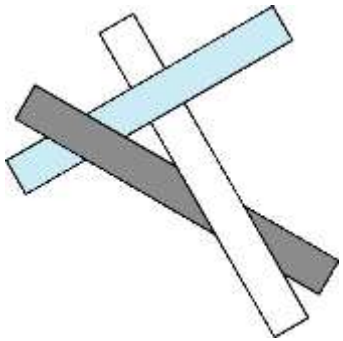


Fill B then A

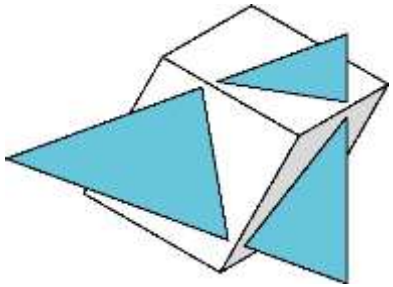
Painter's Algorithm - Hard Cases

No solution for:

- Cyclic ordering
- Intersecting surfaces



cyclic overlap



penetration